ASSIGNMENT 4 – CO4

DESIGNING REUSABLE CLASS HIERARCHIES

Course: Object Oriented Programming with C++ (DSA01)

Course Outcome: CO4 – Analyse and implement C++ applications using inheritance, types of inheritance, virtual base classes, abstract classes, pointers and polymorphism to design reusable and extensible programs (constructors in derived classes, member classes, pointers to objects, the this pointer and pointers to derived classes)

Nature of Assignment: Individual

Duration: One Week

Total Marks: 50

Submission Date: _____

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5 level)

Mandatory Execution and GitHub Submission

- Every student must individually execute every task in this assignment, including all "Complete the Code" programs.
- Create a GitHub repository named DSA01-<RegisterNumber>-Assignment4 and share it with the course faculty (public, or private with the faculty added as a collaborator).
- Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the C++ source files, input files and output screenshots.
- Include a README.md with your name, register number, compiler and version (for example g++ 13), the exact g++ command used to compile (for example g++ -std=c++17 -Wall -Wextra), the steps to run each program, and a short note of the result of each task.
- Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.
- Submit the GitHub link below and on the cover page of your report. An assignment submitted without a working GitHub link will not be evaluated.

GitHub Repository Link: _____

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output with the expected output.

Predict the Output: Write the exact output of the given program before running it. Then run it and explain every difference between your prediction and the actual output, using the C++ rules involved. Predictions written after running the program will not be credited.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

PURPOSE OF THE ASSIGNMENT

This assignment evaluates your ability to design a class hierarchy before writing its code, so that common behaviour is written once and new kinds of objects can be added without changing working classes.

You are expected to demonstrate the following progression:

Interpret the Requirement → Identify Is-a and Has-a Relationships → Design the Hierarchy → Decide Access → Implement → Trace Construction → Challenge → Refine

Where pseudocode is required, it must follow the SIMATS Engineering – Standard for Algorithmic Pseudocode. All code must compile with g++ -std=c++17 -Wall -Wextra.

TASK 1 – CLASS DESIGN REVIEW

Bloom's Level: K5 – Evaluate

Diagnose and Repair Duplicated Classes

6 Marks

A college payroll program has three separate classes written by different developers:

```
class Professor   { public: string name; int id; double basic; double da; double research;
                    double pay() { return basic + basic * 0.42 + research; } };
class LabAssistant{ public: string name; int id; double basic; double da;
                    double pay() { return basic + basic * 0.42; } };
class GuestFaculty{ public: string nm;   int code; double hours; double rate;
                    double pay() { return hours * rate; } };
```

Task 1.1 – Diagnose the Design | 2 Marks

Identify at least five design deficiencies. Examine aspects such as:

- Repeated fields and repeated pay rules
- Public data members
- Inconsistent names for the same information
- The unused da field
- The magic number 0.42
- The effort needed to add a new staff category
- The inability to process all staff in one loop

Task 1.2 – Repair the Design | 3 Marks

Redesign the classes as a hierarchy with a common base class, protected or private data, and a pay() function overridden in each derived class. Draw the UML class diagram.

Task 1.3 – Design Defence | 1 Mark

In not more than 100 words, explain how your hierarchy makes adding a new staff category easier.

Alternative Concept and Complexity Analysis

If the three staff types were represented by one Staff class with a type field and a switch inside pay(), instead of a base class with derived classes, what would the solution be? Analyse the time per pay() call (O(1) in both) and the space per object (unused fields in the single class), and count the places that must change when a new staff type is added.

Evidence Expected

A reasoned diagnosis, a corrected hierarchy with a UML diagram and a concise justification.

TASK 2 – FROM REQUIREMENT TO HIERARCHY

Bloom's Level: K5 – Evaluate

University People Management

8 Marks

A university keeps records of persons (name, ID, phone). Students have a programme and CGPA; staff have a department and salary. A teaching assistant is both a student and a staff member. Research scholars are students who also hold a fellowship.

Task 2.1 – Identify the Relationships | 2 Marks

Classify each relationship as is-a (inheritance) or has-a (member class), and name the type of inheritance (single, multiple, multilevel, hierarchical, hybrid) used for each part.

Task 2.2 – Resolve the Diamond | 2 Marks

Explain the problem that arises when TeachingAssistant inherits from both Student and Staff, and how virtual base classes solve it.

Task 2.3 – Design the Classes | 3 Marks

Write the class declarations with access specifiers, and decide for each base class whether it is inherited publicly, protectedly or privately.

Task 2.4 – Access Table | 1 Mark

Prepare a table showing how a public, protected and private member of Person is visible in Student under public, protected and private inheritance.

Predict the Output

Write the exact output of the program below BEFORE running it. Then compile and run it, and explain every difference between your prediction and the actual output.

```
#include <iostream>


struct Person  { Person()  { std::cout << "Person "; } };

struct Student : virtual Person { Student() { std::cout << "Student "; } };

struct Staff   : virtual Person { Staff()   { std::cout << "Staff "; } };

struct TeachingAssistant : Student, Staff {
   TeachingAssistant() { std::cout << "TA "; }
};


int main()
{
   TeachingAssistant ta;
   std::cout << '\n';
}
```

My predicted output (write before running):

Explain why Person is printed only once. Remove the word virtual from both base classes, predict the new output and compiler messages, and verify.

Alternative Concept and Complexity Analysis

If TeachingAssistant contained a Student object and a Staff object as members (composition, has-a), instead of inheriting from both through virtual base classes, what would the solution be? Analyse the space per object (one or two Person sub-objects) with sizeof, and compare the time and ambiguity of reaching the name field in each design.

Evidence Expected

A relationship analysis, a resolved diamond, class declarations and a correct access table.

TASK 3 – CONSTRUCTORS IN DERIVED CLASSES AND MEMBER CLASSES

Bloom's Level: K5 – Evaluate

Order of Construction and Destruction

9 Marks

Task 3.1 – Pass Values to Base Constructors | 3 Marks

Write constructors for Student, Staff and TeachingAssistant that pass values to their base classes through initializer lists. Explain why the most-derived class must call the virtual base constructor directly.

Task 3.2 – Member Classes | 2 Marks

Add an Address member class to Person and a Course member class to Student. Show how their constructors receive values.

Task 3.3 – Trace the Order | 3 Marks

Print a message in every constructor and destructor and record the order for a Student, a Staff and a TeachingAssistant object. Explain the rule you observe.

Task 3.4 – Experiment | 1 Mark

Remove the virtual keyword from the base classes and record how the output and sizeof(TeachingAssistant) change.

Predict the Output

Write the exact output of the program below BEFORE running it. Then compile and run it, and explain every difference between your prediction and the actual output.

```
#include <iostream>

struct Engine {
   Engine() { std::cout << "Engine "; }
   ~Engine() { std::cout << "~Engine "; }
};

struct Vehicle {
   Vehicle() { std::cout << "Vehicle "; }
   ~Vehicle() { std::cout << "~Vehicle "; }
};

struct Car : Vehicle {
   Engine e;
   Car() { std::cout << "Car "; }
   ~Car() { std::cout << "~Car "; }
};

int main()
{
   {
      Car c;
      std::cout << "| ";
   }
   std::cout << '\n';
}
```

My predicted output (write before running):

State the rule for the order in which base classes, member objects and the derived class are constructed and destroyed.

Complete the Code

Complete TODO 1 below using a member initializer list, and execute the program.

```
#include <iostream>
#include <string>

class Person {
protected:
    std::string name;
    int age;

public:
    Person(const std::string& n, int a) : name(n), age(a) {}
};

class Address {                    // used as a member class (composition)
    std::string city;
    int pin;

public:
    Address(const std::string& c, int p) : city(c), pin(p) {}
    std::string str() const { return city + "-" + std::to_string(pin); }
```

```
};

class Student : public Person {
    long long regNo;
    Address home;

public:
    // TODO 1: Write the constructor: initialise the Person base, regNo and the home
    //   Address member in the initializer list



    void print() const
    {
        std::cout << regNo << ' ' << name << " (" << age << ") " << home.str() << '\n';
    }
};

int main()
{
    Student s("Kavya", 19, 192311001, "Salem", 636001);
    s.print();
}
```

Expected output:

192311001 Kavya (19) Salem-636001

Swap the order of the items in your initializer list, compile with -Wall and record the warning.

Alternative Concept and Complexity Analysis

If Student held a pointer to a dynamically created Address (Address* home = new Address(…)) instead of an Address member object, what would the solution be? Analyse the extra time for allocation and the extra space for the pointer, and state what the constructor, copy constructor and destructor must now do.

Evidence Expected

Correct derived-class constructors, member classes, traced construction and destruction orders and an explained experiment.

TASK 4 – POINTERS TO DERIVED CLASSES: FORENSICS

Bloom's Level: K5 – Evaluate

7 Marks

A bank stores different kinds of accounts through base-class pointers. Senior-citizen accounts should earn 4.0% interest, but every account earns 3.5%, and memory reports show that part of each senior account is never released.

```
class Account { public: double rate() const { return 3.5; } ~Account() {} };

class SeniorAccount : public Account {

   double* history = new double[100];

public: double rate() const { return 4.0; } ~SeniorAccount() { delete[] history; } };


Account* accounts[2] = { new Account, new SeniorAccount };

for (Account* a : accounts) std::cout << a->rate() << "\n";

for (Account* a : accounts) delete a;

Account copy = SeniorAccount();     // stored in a vector<Account> elsewhere
```

Task 4.1 – Locate the Defects | 2 Marks

Identify why the senior rate is never used, why memory leaks, and what object slicing does in the last line.

Task 4.2 – Construct Evidence | 2 Marks

Run the program under -fsanitize=address and add output to the destructors to prove each defect.

Task 4.3 – Repair the Program | 2 Marks

Correct the program using virtual functions, a virtual destructor and pointers (or references) instead of copies.

Task 4.4 – Prevent Recurrence | 1 Mark

State two rules that would have prevented these defects (for example: a base class with virtual functions needs a virtual destructor; use override).

Predict the Output

Write the exact output of the program below BEFORE running it. Then compile and run it, and explain every difference between your prediction and the actual output.

```
#include <iostream>


struct Account {
    double rate() const { return 3.5; }
    virtual double vrate() const { return 3.5; }
    virtual ~Account() = default;
};


struct SeniorAccount : Account {
    double rate() const { return 4.0; }
    double vrate() const override { return 4.0; }
};


int main()
{
    SeniorAccount s;
    Account* p = &s;
    Account copy = s;
    std::cout << p->rate() << ' ' << p->vrate() << ' ' << copy.vrate() << '\n';
}
```

My predicted output (write before running):

Explain each of the three values: which call is bound at compile time, which at run time, and why the copy prints 3.5.

Alternative Concept and Complexity Analysis

If the accounts were stored as std::unique_ptr<Account> in a std::vector, instead of raw pointers deleted by hand, what would the solution be? Analyse whether time or space changes (the same, O(n) to process n accounts) and which of the three defects can still occur.

Evidence Expected

Correct defect identification, sanitizer and destructor evidence, a repaired program and prevention rules.

TASK 5 – POINTERS TO OBJECTS AND THE this POINTER

Bloom's Level: K5 – Evaluate

7 Marks

A grocery delivery app keeps a shopping Cart object for each customer.

Task 5.1 – Four Ways to Reach an Object | 3 Marks

Write code that uses a Cart (a) directly by name, (b) through a pointer with ->, (c) through a reference, and (d) through a pointer created with new and released with delete. Use the this pointer to return *this so that calls can be chained, and to separate a member from a parameter with the same name.

Task 5.2 – Change the Requirement | 2 Marks

The app now keeps an array of pointers to Cart objects for all active customers. Write code that finds the cart with the highest total using pointer arithmetic or indexing.

Task 5.3 – Compare | 2 Marks

Compare the four ways on safety, lifetime and when each should be used. Explain why this is a pointer and what *this refers to.

Complete the Code

Complete TODO 1 and TODO 2 below using the this pointer, and execute the program.

```
#include <iomanip>
#include <iostream>
#include <string>

class Cart {
    std::string items[10];
    double prices[10];
    int n = 0;
    double discount = 0;              // percent

public:
    // TODO 1: add(): store the item and price if there is space, and return *this so
    //   that calls can be chained



    // TODO 2: setDiscount(): use this-> to tell the member apart from the parameter
    //   named discount, and return *this



    double total() const
    {
        double t = 0;
        for (int i = 0; i < n; ++i) t += prices[i];
        return t * (100 - discount) / 100;
    }
};

int main()
{
```

```
    Cart c;
    c.add("Rice", 60).add("Dal", 120).setDiscount(10);
    Cart* p = &c;                    // pointer to an object
    p->add("Oil", 180);
    std::cout << std::fixed << std::setprecision(2) << p->total() << '\n';
}
```

Expected output:

324.00

Explain what would happen if add() returned Cart instead of Cart&.

Alternative Concept and Complexity Analysis

If the carts were kept in a std::vector<Cart> (objects) instead of an array of pointers to Cart objects, what would the solution be? Analyse the time to find the highest total (O(n) in both), the space (objects copied into the vector versus one pointer each) and what happens to pointers into the vector when it grows.

Evidence Expected

Working code for the four ways, method chaining with this, an array-of-pointers search and a reasoned comparison.

TASK 6 – DESIGN CHALLENGE

Bloom's Level: K5 – Evaluate

Vehicle Rental Fleet

13 Marks

A rental company rents bicycles, two-wheelers, cars and electric cars. All vehicles have a registration number and a daily rate. Motor vehicles also have a fuel or charge level and a service due date; cars have a seating capacity; electric cars have a battery capacity and are charged per kWh instead of per litre. Each vehicle computes its rental charge differently. The company needs to rent, return and list vehicles, and to add new vehicle types in future without changing the existing classes.

You are not given the design.

Stage 1 – Interpret the Problem | 2 Marks

Identify the entities, their data and the behaviour that differs between them. State the requirements as testable statements.

Stage 2 – Propose the Hierarchy | 2 Marks

Draw the class hierarchy, showing an abstract base class with at least one pure virtual function, the types of inheritance and any member classes.

Stage 3 – Identify the Critical Design Decisions | 2 Marks

Decide what is placed in each class, which functions are virtual, and how the fleet is stored (an array of base-class pointers).

Stage 4 – Design the Classes | 4 Marks

Present the class declarations and pseudocode for renting and returning a vehicle.

Stage 5 – Implement and Demonstrate | 3 Marks

Implement the program and demonstrate renting each type, returning with a late charge, and adding a new vehicle type (for example an electric bus) without changing the existing classes.

Alternative Concept and Complexity Analysis

If the fleet used a single Vehicle class with a type code and a switch in charge() instead of an abstract base class with virtual functions, what would the solution be? Analyse the time per charge() call (a switch versus a virtual call, both O(1)), the space per object (unused fields versus one vtable pointer), and the classes changed when a new vehicle type is added.

Evidence Expected

A complete hierarchy design with pseudocode, a working program and a demonstrated extension.

Submission Requirements

GitHub repository link (mandatory) containing the executed code for every task, including every completed “Complete the Code” program, with input files, output screenshots and a README.md.

- Report containing diagnoses, UML diagrams, access tables, construction traces, pseudocode and comparisons
- C++ source code for Tasks 1.2, 3, 4, 5 and 6
- Predictions, outputs and sanitizer reports

Assessment Verification

Students may be asked to explain any class, trace the construction of an object, or extend the hierarchy during a brief verification discussion/viva.